Modeling, Simulation and Control Report Group 15

SSY191 — Model-Based Development of Cyber-Physical Systems

Ayman Alzein Muddur Ajay Nayak Armin Khorsandi Mohammad Kassem

***Abstract*—This project focused on an extensive exploration of quadrotor dynamics, sensor integration, control strategies, and modeling using Simulink and Simscape. Beginning with experiments to understand the Crazyflie quadrotor's orientation axes and gyro behavior, the project progressed to data analysis using Simulink and MATLAB, implementing a complementary filter for accurate orientation estimation. Control strategies were developed, including a P controller for motor management based on roll and pitch angles, while Simscape facilitated the creation of a detailed plant model for quadrotor dynamics. Open-loop simulations validated model accuracy, leading to the design and implementation of an LQ-controller for stabilizing roll and pitch angles. This project offers valuable insights into dynamics, sensor fusion, and control theory for practical applications in aerial robotics.**

***Index Terms*—Crazyflie, Simscape, P-controller, LQR**

I. Introduction

Quadrotor control is fundamental to modern aerial robotics, influencing a wide array of applications from surveillance to delivery systems. Achieving precise and stable flight in the quadrotor requires sophisticated control strategies grounded in accurate models of their dynamics. The key components of quadrotor control include managing roll, pitch, and yaw angles, which are influenced by raw sensor data such as accelerometer and gyroscope readings. Model-based design emerges as a critical methodology, offering a structured approach to developing control algorithms by capturing the intricate dynamics of quadrotor systems through simulations and mathematical models.

The fusion of accelerometer and gyroscope readings provides insights into the quadrotor's orientation and angular velocity, forming the foundation for control algorithm development. Understanding the interplay between these sensor inputs and the quadrotor's actuators, such as motors, is crucial for designing control strategies that ensure stability and responsiveness in flight.

In this project, we delve deep into quadrotor control principles, highlighting the significance of model-based design in achieving robust and precise control of the quadrotor. By incorporating raw sensor data, including accelerometer and gyroscope readings, into our simulations and modeling using Simscape for accurate system representation, we aim to showcase the efficacy of this approach in addressing challenges related to quadrotor stabilization, maneuverability, and overall flight control.

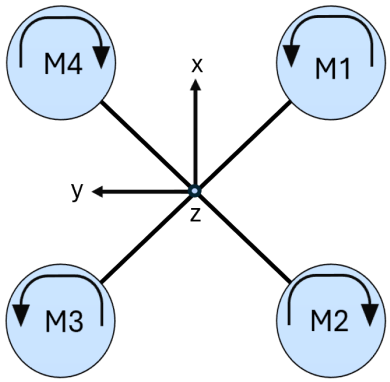


Fig. 1. Orientation of the quadrotor and their rotation

All the references in the following sections are taken from [1].

A. Units

- Time: seconds (s)
- ϕ: Roll angle (rad)
- θ: Pitch angle (rad)
- ψ: Yaw angle (rad)
- Θ: Euler angle
- h: Sampling interval (s)
- α: Tuning parameter in complimentary filter (rad/s)
- T: Thrust (N)
- b: Lift constant $(N/(rad/s)^2)$
- m: Mass (kg)
- p: Position (m)
- v: Velocity (m/s)
- a: Acceleration (m/s^2)
- τ: Torque (N·m)
- k: Drag constant $(N * m/(rad/s)^2)$
- ω: Angular velocity (rad/s)
- d: Arm length of the quadrotor (m)
- J: Angular inertia $(kg * m^2)$
- x: State vector
- u: Control input vector (N)
- K: Feedback gain matrix
- F: Air friction coefficient (N·s/m)

II. Orientation Estimation

Orientation estimation in quadrotor is crucial for precise control and stable flight. Roll, pitch, and yaw are fundamental rotational movements that determine the quadrotor's orientation in three-dimensional space. Angular velocity, measured by gyroscopes, provides information about the rate of change of these angles, while accelerometers measure linear acceleration, including gravitational forces.

Gyroscopes are vital for measuring angular velocity accurately, but they are prone to drift over time due to integration errors. High-pass filters are commonly used to remove low-frequency noise and bias from gyro readings, reducing drift errors. On the other hand, accelerometers provide gravity-related measurements along the three axes, aiding in determining the orientation relative to the gravitational vector. Low-pass filters are often applied to accelerometer data to eliminate high-frequency noise and vibration effects.

Complementary filters combine data from both these sensors to improve orientation estimation. By fusing gyroscope and accelerometer data, complementary filters leverage the high accuracy of gyroscopes at short intervals and the stability of accelerometers over longer durations. This fusion technique reduces drift errors associated with gyroscopes while maintaining stability during dynamic maneuvers, enhancing overall orientation estimation accuracy in quadrotor.

A. Orientation Estimation — Methodology

In the project, we established the coordinate axes of our quadrotor system based on its behavior. Following the right-hand rule when viewed from the top, the x-axis (roll - ϕ) aligns with the forward motion, the y-axis (pitch - θ) points towards the left, and the z-axis (yaw - ψ) extends outward from the surface.

With the coordinate axes determined, we proceed to derive the rotation matrix using the extrinsic rotation order of Roll-Pitch-Yaw, giving us:

$$\begin{aligned} {}^{B}R_{W} &= \left[{}^{W}R_{B}\right]^{T} \\ &= R_X^T(\phi)R_Y^T(\theta)R_Z^T(\psi) \end{aligned}$$

The accelerometer measures the forces acting on the drone's body. It is calibrated to return a value of 1 on the z-axis when the drone is stable on a horizontal surface, like a table. By using the force balance equation and the matrix ${}^{B}R_{W}$, we obtain the following:

$$\underbrace{\begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix}}_{Acc-readings} = m\ ({}^{B}R_{W}) \underbrace{\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}}_{Known forces}$$

$$\begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix} = m \begin{bmatrix} ... & ... & -s(\theta) \\ ... & ... & c(\theta)s(\phi) \\ ... & ... & c(\theta)c(\phi) \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Now we can derive formulas for obtaining roll and pitch angles. We do not need the mass value in the estimation of the angles, as it cancels out in the equation. Therefore, by combining multiple equations from the matrix above, the angle estimation equations will be:

$$\begin{aligned} \phi &= \text{atan2}({}^{B}f_y, {}^{B}f_z) \\ \theta &= \text{atan2}(-{}^{B}f_x, \sqrt{({}^{B}f_y)^2 + ({}^{B}f_z)^2}) \end{aligned}$$

The function atan2 is used in the estimation because, this returns the correct sign for the angle estimation.

Moving on, we implement sensor data fusion using a complementary filter. This filter combines gyroscope data (filtered through a high-pass filter) and accelerometer data (filtered through a low-pass filter). For a cut-off frequency of $1/\alpha$ in Laplace domain, the low-pass filter is represented as:

$$G(s) = \frac{1}{\alpha s + 1}$$

and the high-pass filter is simply $1 - G(s)$.

Using the complementary filter equation:

$$\Theta(s) = G(s)\Theta_a(s) + (1 - G(s))\Theta_g(s)$$

($\Theta_g \ and \ \Theta_a$ are the angle estimations from gyroscope and accelerometer, respectively.)
and discretizing it with Euler-backward formula, we obtain the final formula for the complementary filter in discrete form as:

$$\hat{\Theta}_k = (1 - \gamma)\Theta_{a,k} + \gamma(\hat{\Theta}_{k-1} + h\Theta_{g,k})$$

where h is the sampling interval and $\gamma = \frac{\alpha}{\alpha + h}$. In our project, we set α (see Section V-C) to 5, demonstrating its impact on the filter's behavior in balancing gyroscope and accelerometer contributions.

III. Plant Modeling

The quadrotor's plant modeling in Simulink using the Simscape toolbox follows an object-oriented, declarative approach. Instead of explicitly defining input/output equations, we simply list the equations, and the solver handles the rest.

The Simscape code represents a comprehensive plant model for a quadrotor system, essential for simulating and analyzing its dynamic behavior. The inputs section defines control inputs, specifically the thrust generated by each motor, crucial for controlling the quadrotor's motion. Meanwhile, the outputs section details essential system outputs such as position, velocity, acceleration, angles (roll, pitch, yaw), and angular velocity. These outputs collectively provide valuable insights into the quadrotor's state and dynamics during flight, aiding in performance evaluation and control design.

In the parameters section, key physical properties and constants are defined, including mass, gravity, arm length, lift, drag constants, and air friction coefficients. These parameters play a vital role in calculating forces, torques, and other dynamic quantities that influence the quadrotor's motion and stability.

The heart of the model lies in the equations section, where dynamic equations governing the quadrotor's behavior are articulated. This includes transformations between coordinate

frames (e.g., body frame and world frame), calculations of forces and torques based on control inputs, and dynamic equations for angular velocity and acceleration. The code encapsulates computations for thrust, torque, and rotational motion, factoring in the quadrotor's mass distribution and inertial properties. These equations help simulate how the quadrotor flies, allowing for detailed analysis and the creation of control algorithms for the project.

A. Plant Modeling — Methodology

For a non-zero rotor speed ω, the upward thrust vector is given by:

$$T = b\omega^2$$

where b is the lift constant.

Considering the translational dynamics in the world coordinate frame using Newton's second law of motion, we have:

$$m\dot{v} = \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} - {}^{W}R_B \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} - BV$$
$$\dot{v} = a$$
$$\dot{p} = v$$

Here, m is mass, a is acceleration, v is velocity, p is position, T is total thrust, and B is the aerodynamic friction constant.

The torque applied to each motor, opposed to aerodynamic drag, is given by:

$$\tau = k\omega^2$$

where k is the drag constant.

Total torque in x, y, and z directions is calculated as:

$$\tau_z = -\tau_1 + \tau_2 - \tau_3 + \tau_4$$
$$\tau_x = d\cos(\pi/4)(T_3 + T_4 - T_1 - T_2)$$
$$\tau_y = d\cos(\pi/4)(T_2 + T_3 - T_1 - T_4)$$

Here, $d\cos(\pi/4)$ gives the distance between the quadrotor's center and propellers in the x and y directions.

The quadrotor's rotational accelerations are governed by the Euler equation in the body frame:

$$J \cdot \dot{\omega} = -\omega \times (J \cdot \omega) + \Gamma$$

where Γ is the torque applied to the quadrotor, given by: $\Gamma = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}$. Additionally, angular velocities are transformed from the body frame to the world frame.

IV. Plant Linearization

Linearization of the plant model is a crucial step in preparing it for control design, as it allows us to approximate the nonlinear dynamics of the system around an operating point. This process is necessary because many control design techniques, such as linear quadratic regulators (LQR) or PID controllers, are based on linear models. By linearizing the model, we can analyze the system's stability, controllability, and performance using classical control theory tools.

To linearize the plant model, we first need to choose a suitable operating point. This point represents the nominal conditions under which the system operates and is crucial for deriving a valid linearized model. In our case, we selected roll, angular velocity of roll, pitch, angular velocity of pitch, yaw and angular velocity of yaw as states for the linearized model.

Using the selected operating point, the linearized model is computed analytically. This involved linearizing the nonlinear equations describing the system's dynamics around the operating point by taking partial derivatives. The resulting linearized model provides a linear approximation of the system's behavior around the chosen operating point, facilitating the design and analysis of control strategies that can stabilize and regulate the quadrotor's motion effectively.

A. Plant Linearization — Methodology

In general, the model can be described by the following system of equations:

$$\dot{x} = f(x, u)$$

where, f can be any non-linear function of states x and control inputs u.

The linearized model from the above equation can be derived as follows:

$$\dot{x} \approx \left.\frac{\partial f}{\partial x}\right|_{x_0,u_0} \Delta x + \left.\frac{\partial f}{\partial u}\right|_{x_0,u_0} \Delta u$$

where, x_0 and u_0 are the operating points which are taken to be around the point where the angles and angular velocities are zero.

Using this formulation, we get the following A and B matrices:

$$A_{\text{lin}} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B_{\text{lin}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -2.8376 & -2.8376 & 2.8376 & 2.8376 \\ 0 & 0 & 0 & 0 \\ -1.9141 & 1.9141 & 1.9141 & -1.9141 \\ 0 & 0 & 0 & 0 \\ -0.4282 & 0.4282 & -0.4282 & 0.4282 \end{bmatrix} \times 10^3$$

V. Design of the Linear Quadratic Controller

To design an LQ-controller for stabilizing the quadrotor pose in roll and pitch angles around zero, we first define our control objective and constraints. The LQ-controller aims to minimize a cost function that penalizes deviations from the desired roll and pitch angles while considering system dynamics and control inputs.

The design process involves several key steps.
1. **System Modeling:** Develop a mathematical model of the quadrotor dynamics, including state-space representation for roll and pitch angles.
2. **Cost Function Formulation:** Define a cost function that quantifies the control effort and deviation from desired angles. The choice of weights in the cost function reflects the trade-offs between performance and control effort.
3. **LQR Design:** Use the Linear Quadratic Regulator (LQR) method to design the controller. LQR optimally computes control gains by minimizing the cost function, resulting in a state feedback controller that stabilizes the system.

In our scenario, we excluded yaw from the state vector due to the inherent difficulty in estimating yaw using only accelerometer and gyro data. We prioritize stabilizing roll and pitch angles, as they are crucial for maintaining quadrotor stability during flight. While controlling yaw rate can enhance flight performance, it is not a primary requirement for this specific controller. Parameter tuning in the LQR design involves adjusting weights in the cost function to balance control efforts and stability margins. Simulation-based testing validates the controller's performance under various conditions, refining it as needed for real-world application.

A. Methodology

Taking the linearized matrices, we now discretize them, for which the sampling time is taken to be 0.01s. As the input is assumed to piece-wise constant, we employ the zero-order hold method. Using the inbuilt MATLAB function - `c2d`, we derive the discretized matrices as the following:

$$A = \begin{bmatrix} 1.0000 & 0.0100 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.0100 & 0 \\ 0 & 0 & 0 & 1.0000 & 0 \\ 0 & 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

$$B = \begin{bmatrix} -0.1419 & -0.1419 & 0.1419 & 0.1419 \\ -28.3756 & -28.3756 & 28.3756 & 28.3756 \\ -0.0957 & 0.0957 & 0.0957 & -0.0957 \\ -19.1414 & 19.1414 & 19.1414 & -19.1414 \\ -4.2821 & 4.2821 & -4.2821 & 4.2821 \end{bmatrix}$$

Having discretized the model, we have completed the first step in the design process of the LQR. The second step would be to tune and determine the optimal cost matrices Q and R (see Section V-C) in order to minimize the cost function:

$$J = \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k)$$

Again, using the built-in discrete LQR function in MATLAB — `dlqr`, we calculate the gain matrix K, which turns out to be:

$$K = \begin{bmatrix} -0.0088 & -0.0089 & -0.0130 & -0.0131 & -0.0576 \\ -0.0088 & -0.0089 & 0.0130 & 0.0131 & 0.0576 \\ 0.0088 & 0.0089 & 0.0130 & 0.0131 & -0.0576 \\ 0.0088 & 0.0089 & -0.0130 & -0.0131 & 0.0576 \end{bmatrix}$$

The Q and R matrices need to be tuned and tried multiple times until a desired gain matrix K is derived, and the optimal behavior is seen in the overall system.As we adjust the value of Q, we adjust the penalty for state deviations from the desired set point. This results in a system that reacts more quickly and aggressively to correct any deviations. Conversely, when we increase some values in R, we penalize more some control input from the other. This tends to make our system more cautious and conservative.

The goal is to incorporate a base thrust into the quadrotor's motor control system. This base thrust acts as a feed-forward controller to counteract the effects of gravity. Consequently, the total control input applied to the quadrotor can be expressed as follows:

In the following equation:

$$u(k) = -Ke(k) * (1/pwm) + u_{\text{basethrust}}(k)$$

- $u(k)$ represents the total control input at time step k.
- $x(k)$ corresponds to the state vector of the quadrotor.
- $r(k)$ corresponds to the states reference values.
- $e(k)$ corresponds to the deviation of $x(k)$ from $r(k)$.
- K is the gain matrix used for feedback control.
- $u_{\text{basethrust}}(k)$ represents the desired base thrust.
- pwm represents conversion from motor signal to force.

B. Simulink Model

Figure 2 illustrates the comprehensive schematic of the closed-loop system being implemented. The process commences with input of the accelerometer and gyroscope. Notably, gyroscope readings, initially in radians, are converted into degrees to ensure consistency across the system. The implementation of the complementary filter, as detailed in Section 2 of this report, involves combining data from both sensors to estimate roll (θ) and pitch (ϕ) angles using 'atan2' formulas, as previously discussed.

The inputs to the LQR controller consist of estimated roll and pitch angles from the complementary filter, along with angular velocities in the x, y, and z directions of the gyroscope. These values are then compared with the respective reference values, and the resulting vector is multiplied by the feedback gain matrix derived from the discrete LQR function. This process yields the final output from the LQR controller, representing the thrust command.

Subsequently, the thrust values are adjusted relative to the base thrust values before being sent to the motors for execution to ensure that the drone will not climb and stuck to the roof during the physical testing.

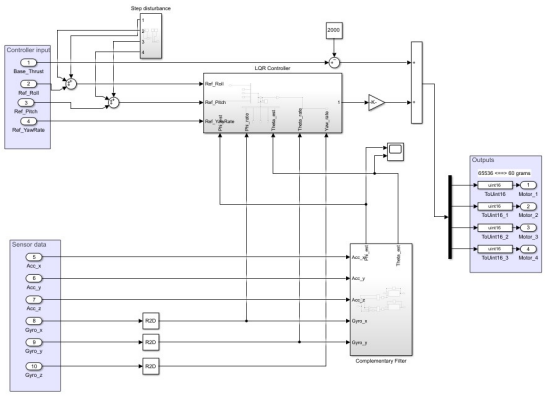


Fig. 2. Simulink model of the closed loop system

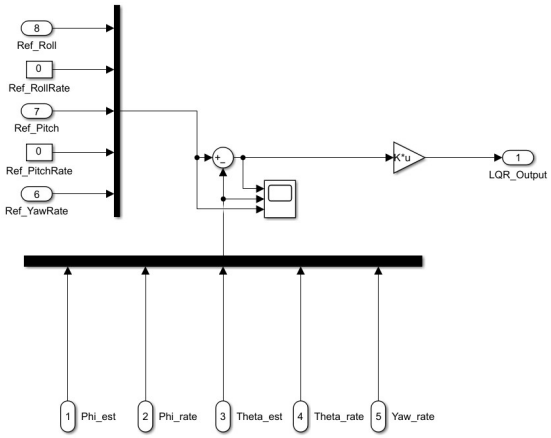


Fig. 3. LQR Controller

C. Discussion on the chosen parameters and methods

The final motivation is to have a stable flight of the quadrotor, and therefore there are several parameters which require tuning to achieve the final goal. These parameters include — ratio alpha, disturbance matrices Q and R, feedback gain matrix K, and the base thrust value. Each of them are explained in detail as follows:

- **Alpha:** The term "$1/\alpha$" represents the cut-off frequency in the complimentary filter. This balances the trust we have on the data coming from the two sensors. Meaning, a larger alpha results in a low cut-off frequency and more contribution will be from the gyroscope. On the other hand, smaller alpha would result in a high cut-off frequency and significant contribution from accelerometer. We have taken the alpha value to be 5, resulting in a gamma of 0.998 and thus trusting the gyroscope more, mainly to reduce the noise in the estimation of roll and pitch angles.
- **Q and R matrices:** Higher values in the diagonal elements of the Q matrix prioritize minimizing deviations in specific state variables, such as roll and pitch angles, leading to more aggressive control actions in response to deviations from desired values. On the other hand, lower values allow more leeway in these deviations but may result in smoother control actions. In our case, we wanted to control the pitch and the roll angles on the drone, so we could have penalized them more than the angular velocities. This results in more aggressive responses that compensate for the angles' error faster in the simulation. However, the real drone does not match fully the mathematical model that we are simulating. Having a more aggressive control signal will decrease the robustness of the controller, and therefore the drone will be less stable than what we see in the simulation. Tests were conducted by initially increasing the penalty for the roll and pitch angles and then uploading the code to the actual drone. Under these conditions, the drone struggled to stabilize itself. However, when the Q matrix was set to an identity matrix, the drone's stability significantly improved, allowing it to fly effectively.

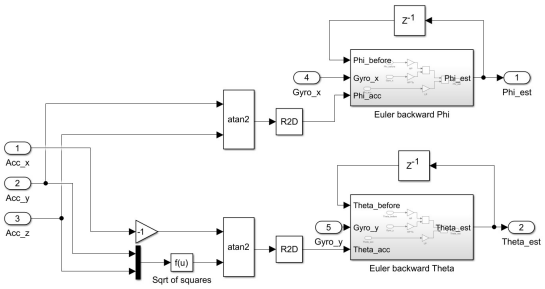


Fig. 4. Complimentary filter

In contrast, the R matrix in the LQR controller influences how control efforts are utilized in the optimization process. Higher values for some control signals in R encourage higher penalties of these control inputs in the cost of the other control signals. Since we have 4 actuators and all of them are equally expensive to run, we will penalize them equally in the R matrix. As a result, we assigned an identity matrix to the R.

Below are the Q and R matrices that we have taken. These specific Q and R matrices were finalized after observing a stable response in the real-world scenario, despite the controller's slow response in the simulations.

$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Gain matrix K:** Changes in the gain matrix K directly impact the LQR controller's output by determining how the control inputs are calculated based on the state error. Adjusting the values in the Q and R matrices can fine-tune

the behavior of the controller, balancing between achieving desired system performance and minimizing control effort or actuator activity. Therefore, careful tuning of Q and R matrices and understanding their impact on K is crucial for optimizing the performance of the LQR controller in real-world applications.

- **Base thrust value:** In order for the drone to stay in midair, it should generate a thrust equal to the gravitational force, so the net force applied to the drone becomes zero. The base thrust value is written in terms of motor signal. The motor signal value will be between 0 and 65536 which corresponds to 0% and 100%,respectively, of the motor power. The value of base thrust was derived after multiple rounds of experimental tuning to be 23000.

VI. Evaluation of Control Design

The evaluation of the control design implemented for the quadrotor primarily focused on assessing the effectiveness and efficiency of the Linear Quadratic Regulator (LQR) in stabilizing roll and pitch angles. Various metrics and tests were employed to validate the performance of the control system under simulated conditions and anticipated real-world scenarios.

A. Simulation Results

Extensive simulations were conducted to analyze the performance of the LQR controller in maintaining the desired orientation of the quadrotor. The simulations were aimed at evaluating the response of the quadrotor to disturbances, its ability to recover from initial angular displacements, and the overall stability.

- **Response to Disturbances:** The controller was tested against a specific type of disturbance, which is the impulse disturbance. The quadrotor equipped with the LQR demonstrated robustness by quickly stabilizing to the desired orientation, showing minimal overshoot and rapid settling time. This test illustrates the controller's capability to handle sudden and sharp disturbances, which are typical in real-world scenarios where quick dynamic responses are essential.
- **Recovery from Initial Displacements:** Simulations were conducted with the quadrotor starting from non-zero roll and pitch angles. The simulations showed that the controller efficiently brought the system back to equilibrium within a few seconds, confirming its effectiveness in critical recovery scenarios. However, in real-world tests, the quadrotor was slightly shaky but still managed to maintain balance.

B. Behavior of the quadrotor in the real world scenario

After successfully simulating the filter and controller in Simulink with promising results, the code was deployed onto the quadrotor for real-world testing. The quadrotor was suspended vertically using a fishing wire. During the tests, the quadrotor exhibited expected behavior but experienced slight oscillations. Despite this, it effectively maintained its balance in the air, even when subjected to disturbances such as gentle pulls from the fishing wire and gusts of air directed towards it, as well as tilting in various directions. After successfully maintaining its balance with the assistance of fishing wire, the quadrotor was subsequently tested without the wire. The quadrotor demonstrated stability, maintaining its balance and flying for several seconds before the test was concluded and the quadrotor was safely intercepted. A video recording of that specific test is available for viewing here. [2]

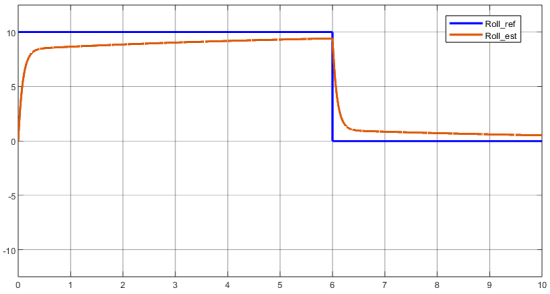


Fig. 5. Simulation graph of the roll angle against the disturbance

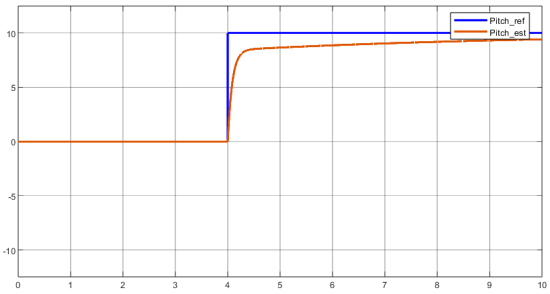


Fig. 6. Simulation graph of the pitch angle against the disturbance

C. Practical Implementation Considerations

While the simulation results were promising, practical implementation considerations were discussed to ensure the transition from theory to practice. These considerations are crucial, as simulations, while useful, often do not capture all the complexities of the real world. Differences between simulated environments and actual conditions can lead to discrepancies in performance. As such, it is not sufficient to tune the LQR controller solely based on simulation result

D. Plots from the simulation

Figures 5 and 6 illustrate the results obtained from the LQR, depicting the system's response to a step disturbance and the LQR's output for the roll and pitch angles. The plots clearly demonstrate the effectiveness of the designed controller. The response to the disturbance is prompt, and the system returns to its normal state once the step disturbance is removed, effectively attenuating deviations and bringing the roll and pitch angles back to the reference values.

VII. Implementation using C

This part of the project aims to provide experience in implementing filters and control algorithms using C, and to work with a real-time operating system (RTOS) to create and coordinate concurrent tasks using semaphores. Throughout the project, we have used Matlab, Simscape, and Simulink to design and test a complementary filter and LQ-controller for the Crazyflie quadrotor. This assignment involves translating our controller design into C-code.

The project structure includes several key components: a Makefile for building the project, ‘constants.c,h‘ for simulation constants, ‘main.c‘ for dynamics simulation and logging tasks, and ‘reference_generator.c,h‘ for generating roll and pitch reference signals. Additionally, ‘filter.c,h‘ contains a task for implementing the complementary filter, and ‘controller.c,h‘ contains a task for implementing the LQR. The simulation outputs data to the terminal and generates a ‘simulation.csv‘ file for further analysis in Matlab.

Finally, once the filter and LQR implementation produce satisfactory results, the codes are combined into a single file for the Crazyflie. The performance is then checked to ensure it matches the performance of the code generated from Simulink.

A. Filter code in C

The ‘filterTask‘ function is designed to run as a real-time task that estimates the roll, pitch, and yaw angles of the Crazyflie quadrotor using a complementary filter. Initially, the task waits for the system to start and for the sensors to be calibrated. It then enters an infinite loop where it processes sensor data at a fixed rate. The task begins by waiting for the sensors to signal that new data is available, then acquires accelerometer and gyroscope data. The accelerometer data is used to compute the roll and pitch angles, which are then converted to degrees.

To fuse the accelerometer and gyroscope data, the task uses a complementary filter. This filter combines the short-term precision of the gyroscope with the long-term stability of the accelerometer. The roll and pitch angles are estimated by blending these measurements, while the yaw angle is integrated from the gyroscope data. These estimates are then stored in the global state, protected by semaphores to ensure thread safety. The estimated angles are also logged for monitoring. The task runs periodically, controlled by a delay to maintain the desired execution rate.

B. Controller code in C

The ‘controllerTask‘ function is designed to run as a real-time task that stabilizes the Crazyflie quadrotor using a Linear Quadratic Regulator (LQR) controller. After ensuring that the sensors are calibrated, the task initializes necessary variables and enters an infinite loop. Within the loop, it calculates the error between the desired state (assumed to be zero for roll, pitch, and yaw) and the current state obtained from sensor data. This error is computed for both the attitude (roll, pitch, yaw) and the corresponding rates from the gyroscope.

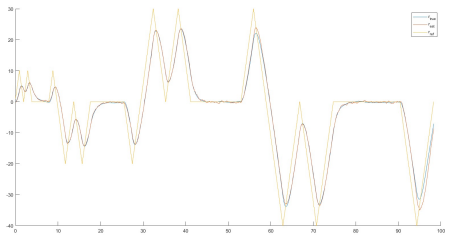


Fig. 7. Plot for the roll angle simulation in C

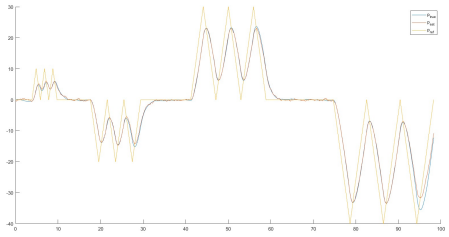


Fig. 8. Plot for the pitch angle simulation in C

The task then applies the LQR control law using pre-computed gains stored in the matrix ‘K‘. These gains were derived from Matlab simulations. The control law calculates the appropriate motor outputs to minimize the error, ensuring stability and control of the quadrotor. The motor commands are bounded to ensure they stay within valid ranges (0 to 65536) and are then sent to the motors. This process is repeated at a fixed rate to maintain stable flight, with the task pausing briefly between iterations to control the execution rate.

Figures 7 and 8 display the simulation results of the filter and controller from the C implementation. In both cases, the estimations closely follow the true signals, confirming the effective performance of both the filter and the controller.

C. Key points for implementation in C

- **Use of Semaphores:**
  Semaphores are used to ensure safe and synchronized access to shared resources among concurrent tasks running in the real-time operating system (RTOS). Specifically, semaphores guard critical sections where sensor data and state estimates are read and updated. For example, when the ‘filterTask‘ reads sensor data or updates the attitude estimates, it uses ‘xSemaphoreTake‘ to lock the relevant semaphore, preventing other tasks from accessing these shared variables simultaneously. Similarly, the ‘controllerTask‘ uses semaphores to safely read the state estimates and sensor data before computing control actions. This coordination prevents data corruption and ensures consistent and reliable operation of the quadrotor control system.
- **Use of vTaskDelayUntil:**
  The ‘vTaskDelayUntil‘ function is preferred over ‘vTaskDelay‘ for implementing periodic tasks in a real-

time system because it offers more precise timing control and helps maintain task execution at regular intervals. While ‘vTaskDelay‘ simply pauses the task for a specified duration, ‘vTaskDelayUntil‘ allows the task to block until a specific future time, ensuring a consistent execution period regardless of the time spent executing other tasks or performing other operations. This precise timing is crucial for tasks that need to meet strict timing requirements, such as controlling the dynamics of a quadrotor. By using ‘vTaskDelayUntil‘, we can guarantee that periodic tasks execute with high accuracy and reliability, minimizing timing jitter and ensuring stable system behavior.

D. Behavior of the quadrotor in the real world scenario

After achieving satisfactory results in the simulations of the filter and LQR (Figures 7 and 8), the codes were combined into a single file for the Crazyflie quadrotor. The code was then loaded onto the quadrotor, which was secured with fish wires.

The C code used the same parameters, including the K gain matrix, as the MATLAB simulation. However, the quadrotor was not as stable as it was when running the code generated from MATLAB.This unexpected discrepancy is presumed to stem from differences in the code creation process.

VIII. Conclusion

In conclusion, this project embarked on a comprehensive exploration of quadrotor dynamics, sensor integration, and control strategies, culminating in the implementation of a Linear Quadratic Regulator (LQR) for stabilizing roll and pitch angles. Through simulations and practical implementation using C, we successfully demonstrated the efficacy of the LQR controller in maintaining desired orientation under various conditions.

Our findings underscored the importance of sensor fusion techniques, particularly the complementary filter, in achieving accurate orientation estimation by leveraging the strengths of both gyroscopes and accelerometers. Furthermore, the linearization of the plant model facilitated the application of classical control theory, enabling the design of an LQR controller that efficiently stabilized the quadrotor.

Lessons learned from this project extend beyond theoretical understanding to practical considerations in real-world implementation. We navigated challenges such as discrepancies between simulation and reality, underscoring the need for careful tuning and validation in physical environments. Additionally, the integration of concurrent tasks and synchronization using semaphores in C programming for real-time systems provided valuable insights into software-hardware interaction and system robustness.

Looking ahead, there are several avenues for further research and improvement. Refinement of control algorithms to address yaw stabilization and enhance overall flight performance remains a promising area. Additionally, exploring advanced sensor fusion techniques, such as Kalman filtering, could lead to even more robust orientation estimation in dynamic environments.

Overall, this project not only deepened our understanding of quadrotor dynamics and control theory but also equipped us with practical skills in simulation, software development, and real-time systems. By bridging theory and practice, we have laid a foundation for future advancements in aerial robotics and autonomous systems.

References

[1] “Text references,” May 2024, .All the matrices, formulas, statements, and definitions are taken from the lecture notes and slides that are available on the Canvas page of the course. In addition, certain points were adopted from the consultation hours and discussion sessions held with the TAs.

[2] “Balancing test,” May 2024, https://youtube.com/shorts/SEQ70XpADtU.